

Chapter 1. Introduction

Biology is increasingly becoming a data-driven science, and deep learning—a powerful subfield of machine learning—is opening new ways to uncover patterns in complex, high-dimensional datasets. As these two fields converge, new opportunities are emerging to extract meaningful insights using modern computational tools. This book is a practical introduction to working at that intersection, focused on developing the skills and mindset needed to apply deep learning effectively in biological contexts.

Getting Started

This opening chapter helps you get oriented. Before jumping into code, we walk through how to frame a project, evaluate your data, and avoid common pitfalls. A bit of structure and planning up front will make your work more reproducible, more flexible, and ultimately more useful and impactful.

Deciding What Your Model Will Replace

The success of a deep learning project in biology often hinges on what happens before you write a single line of code. It's easy to get lost in technical details or spend weeks exploring data and architecture variants that don't lead to meaningful outcomes. Especially in a field as interesting as this one, the temptation to tinker is strong. To stay focused, it helps to ask a few grounding questions up front.

One of the most useful is: *What existing process will my model replace or improve?* The most impactful projects in this field often (though not always) have a clear answer. Here are some examples across different domains:

In healthcare and drug discovery:

Skin cancer classification models

Aim to replicate a dermatologist's ability to visually diagnose melanoma or other lesions from clinical images, offering faster, more scalable screening for at-risk populations

Pathogen detection systems

Trained on sequencing data or imaging to detect bacterial or viral infections directly from raw clinical samples (e.g., blood, saliva, or tissue), potentially replacing slower, culture-based diagnostics

Brain tumor segmentation models

Automate or accelerate the process of outlining tumors on MRI scans, a task that radiologists typically perform manually and with great time investment

Drug-target interaction prediction tools

Aim to prioritize the most promising compound-target pairs, reducing the need for costly wet-lab screening of massive chemical libraries

Antibiotic resistance prediction models

Forecast whether a given bacterial strain will resist certain treatments, helping clinicians select effective antibiotics more quickly

In molecular biology:

AlphaFold

This protein structure prediction model, in many cases, replaces the need for experimentally determining 3D protein shapes via expensive lab-based techniques like X-ray crystallography, cryo-EM, or NMR

Gene expression prediction models

Forecast gene activity from raw genomic sequences, offering a computational alternative to RNA sequencing (RNA-seq) experiments

Variant effect prediction models

Help automate the interpretation of genetic mutations, supporting clinical decision making by prioritizing likely pathogenic variants for follow-up analysis or experimental validation

And in ecology and environmental science:

Acoustic species classification systems

Use forest sound recordings to identify animal species present, offering a scalable and less labor-intensive alternative to in-person biodiversity surveys

Crop disease detection

Via drone or satellite imagery, enables early identification of plant stress, reducing the need for manual scouting across large fields

Animal facial recognition tools

Track individual animals, reptiles, birds, and mammals over time without the need for tagging, collars, or other invasive methods

Poaching detection systems

Trained on infrared or motion sensor data, can automatically flag human activity in protected wildlife zones, assisting conservation efforts

Ideally this gives you a flavor of the kinds of workflows deep learning can improve or even replace. Where possible, try to estimate the potential impact of your model—how much time, cost, or manual labor it could save, or what new insights it might enable. This will help you stay grounded in the real-world utility of your work and communicate its value to collaborators, stakeholders, or the public.

That said, not every valuable model needs to replace an existing process. Some open up entirely new capabilities—like generating novel biological sequences, uncovering hidden patterns in large datasets, or linking data types that were never connected before. These models might not streamline a lab task, but they can enable new kinds of discovery, expand what questions we can ask, or offer fresh ways to interpret complex systems. If your model creates something new, just be clear about what that is and why it matters—and be thoughtful about how you evaluate success when no established benchmark exists.

Determining Your Criteria for Success

It's important to define, as early and explicitly as possible, what success looks like for your project. Research can be time-consuming and open-ended, so clear goals help you stay focused and avoid endless tweaking—repeatedly changing models, architectures, or training settings without a clear hypothesis or evaluation plan. This kind of trial-and-error loop is common in deep learning due to the large number of design choices and hyperparameters. Without structure, it can waste time and produce results that are hard to interpret or reproduce.

Examples of success criteria include:

Performance metric (e.g., accuracy, AUC, F1)

You might aim to match the performance of a human expert, achieve a correlation with experimental results comparable to a technical replicate, or keep the false-positive rate below a certain number.

Level of interpretability

In many applications, it's important not only that a model performs well, but also that its decisions can be understood by domain experts. For instance, you may prioritize well-calibrated uncertainty estimates or interpretable feature attributions, especially when trust and explainability are critical.

Model size or inference latency

If your model needs to operate in a resource-constrained environment (e.g., smartphones or embedded devices) or meet real-time throughput targets (e.g., process 20 frames per second), your success criterion might focus on efficiency—such as achieving high performance per floating point operation (FLOP), which measures how effectively the model uses computational resources. In such cases, metrics like inference time, memory usage, or energy consumption may matter more than raw accuracy.

Training time and efficiency

When compute is limited—or for educational contexts—you may prioritize fast training or minimal hardware requirements. Since training deep learning models typically involves large matrix operations, they are often accelerated using graphics processing units (GPUs). In low-resource settings, developing a simpler model that trains quickly on a CPU may be a more practical goal than maximizing performance.

Generalizability

In some cases, the goal is to build a model that works well across many datasets or tasks, rather than one that is finely tuned to a single benchmark. For example, *foundational models*—large models trained on broad datasets that can be adapted to many downstream applications—prioritize flexibility and reuse. In such settings, broad applicability may be more valuable than squeezing out the best possible performance on a specific task.

Defining these goals up front helps you answer the key question: *When is the project done?* You'll likely need to balance multiple criteria—but having them laid out early will keep your efforts aligned and your scope realistic.

Invest Heavily in Evaluations

Once you've defined your criteria for success, it's time to prioritize *evaluation*. This means thinking carefully about precisely how you'll measure progress—including what metrics you'll use, how you'll validate results,

and which baselines you'll compare against. Without a clear, well-designed evaluation strategy, even a technically impressive model can fail to produce meaningful conclusions.

Strong evaluations don't just help you measure progress. They also help you detect bugs, estimate task difficulty, and build intuition. The key idea is simple: you need a known point of comparison to understand if your model is doing anything meaningful.

TIP

While no general rule exists, it wouldn't be surprising if successful machine learning projects spent 50% of their time designing evaluation strategies and running baselines, 25% curating or processing data, and only 25% on model architecture. Without good evaluations, you're flying blind: you won't know whether your model is actually improving, what trade-offs you're making, or even whether it's learning anything meaningful at all.

Spend time here. Evaluation isn't just something you do at the end. It's something you design at the beginning, and it guides the entire project.

Designing Baselines

One of the most practical evaluation tools is a strong *baseline*—a simple method that gives you something to beat. Good baselines help you measure progress, catch bugs early, and understand the difficulty of your task. Sometimes they can even be surprisingly hard to beat.

Designing good baselines requires thinking carefully about the task. Here are a few common baseline strategies for classification tasks:

Random prediction

Assign labels completely at random, with equal probability for each class. This tells you what performance looks like with no information at all.

Random prediction weighted by class frequencies

Sample labels randomly, but in proportion to how often they occur in the training data. This is useful for imbalanced datasets.

Majority class

Always predict the most common class. This can be a surprisingly hard baseline to beat in highly class imbalanced settings.

Nearest neighbor

Predict the label of the most similar example in the training data (e.g., 1-nearest neighbor using Euclidean distance). This is often effective when inputs are low dimensional or well structured.

And for regression tasks:

Mean or median of the target

Always predict the average or median target value from the training set. This often matches what a model would do if it's not learning anything meaningful.

Linear regression with a single feature

Fit a line using just the strongest individual predictor (e.g., one biomarker). This helps gauge how much a more complex model improves over a simple signal.

K-nearest neighbor regression

Predict the target as the average (or weighted average) of the k most similar data points. This is simple to implement and often surprisingly competitive on structured datasets.

And for both:

Simple heuristics

Use straightforward rules based on domain knowledge. For example, in diagnostics, classify a patient as positive if a single biomarker or measurement exceeds a threshold. For skin cancer images, rank lesions by average pixel intensity. In genomics, if the

task is to predict which gene a mutation affects, a simple baseline is to assume it affects the nearest gene in the genome.

WARNING

If your model can't beat a basic baseline, something is likely off—and that's useful to know. It's a key signal to revisit your data, features, or modeling approach.

Time-Boxing Your Project

It's important to time-box your project—that is, set a fixed amount of time to work on it, after which you pause or stop regardless of the outcome. Many research ideas “fail” in the sense that they don't achieve the desired metrics. That's normal. All projects, even the unsuccessful ones, generate insights that inform future work. Time-boxing helps ensure that failed experiments still move you forward—without consuming unlimited time and energy.

NOTE

Time-boxing doesn't mean giving up easily—it means setting boundaries to maintain focus, avoid burnout, and keep making progress.

Here are some tips for time-boxing effectively:

1. *Set a clear deadline*

Choose a realistic time frame (e.g., two weeks, three months) and stick to it.

2. *Define checkpoints*

Identify intermediate milestones—like completing dataset preprocessing, training a baseline model, or hitting a certain accuracy—to track progress.

3. *Reflect at the end*

Take time to evaluate what worked, what didn't, and what you learned.

TIP

Time-boxing is also useful *within* a project. For example: “I’ll experiment with this new processing or modeling idea for one week, and if it doesn’t help, I’ll move on.”

The biggest risk with time-boxing is yourself. It’s easy to justify extensions, add new ideas, or convince yourself you’ll strike gold if you just try 10 more things. Scope creep and perfectionism are common traps. In these cases, it can help to talk to someone else—a collaborator, mentor, or friend—to get perspective and avoid spinning your wheels. A quick conversation can often cut through indecision or obsessiveness and help you refocus on the broader context.

Deciding Whether You Really Need Deep Learning

This might seem like strange advice in a deep learning book, but before diving in, take a moment to ask yourself: *Do I actually need deep learning for this problem?* We’ll say it again—seriously consider simpler approaches.

Deep learning models are powerful (and undeniably interesting), but they’re also resource intensive, complex to train, and difficult to debug. In many cases, traditional methods—like linear regression, decision trees, or basic statistical techniques—can achieve your goals with far less effort. These approaches are often:

Easier to implement

Quicker to set up and require less expertise

Less computationally demanding

Can run on standard hardware with minimal training time

More interpretable

Easier to explain, troubleshoot, and validate

Carefully weigh the trade-offs. If a simpler method delivers the insights or performance you need, it's often the smarter and more efficient path.

Ensuring That You Have Enough Good Data

Deep learning models don't just need a lot of data—they also generally need high-quality data. Models trained on poor data can often fail catastrophically, regardless of their sophistication.

Make sure you have:

Sufficient quantity

Deep models typically need thousands of examples or more. What counts as “enough” data depends on your problem and architecture. Check relevant literature for benchmarks. If you're working with a small dataset, consider *transfer learning*, where you start from a model trained on a related task and fine-tune it on your own data. This approach can dramatically reduce the amount of data needed to achieve good performance.

Sufficient quality

Clean, consistent data is critical. Label errors, noise, or inconsistencies can seriously degrade performance. Even large language models—like those powering modern chat-based assistant systems—can benefit substantially from training on carefully curated, high-quality data. Prioritize quality checks and thoughtful curation.

Assembling a Team

Working alone is totally fine—but teaming up can accelerate progress, improve your ideas, and make the process more enjoyable. Here are some tips for finding great collaborators:

Engage with the community

Join relevant forums, online groups, or webinars to connect with others, share ideas, and discover potential collaborators. Communities like Reddit, Discord servers, X, and specialized Slack groups can be great starting points.

Participate in hackathons and competitions

Platforms like Kaggle, Zindi, or local university events offer structured challenges, feedback, and opportunities to meet people with similar interests.

Form an interdisciplinary team

Combining expertise from different areas often leads to stronger projects. If you're a biologist, team up with someone in machine learning, and vice versa.

Collaborate with experts

Domain experts can help shape your approach and identify blind spots early. Look for collaborators at conferences or workshops, or reach out to authors of relevant papers. Strangers are surprisingly responsive to genuine cold requests from interested people.

And here are some tips for once you find people to work with:

Define clear goals and roles

When working with others, it helps to clarify responsibilities early—who's doing what, what success looks like, and how decisions will be made. This avoids misunderstandings and keeps the project moving.

Use shared tools for collaboration

Version control (like Git), shared notebooks (e.g., Google Colab), and simple task trackers (like Notion or Trello, or just a shared Google Doc with some lists) can make it much easier to coordinate and stay organized.

Support specialization

Let people lean into the parts of the project they enjoy most—some may focus on infrastructure and software engineering, others on data curation, modeling, or biological interpretation.

Start small

If you're unsure about long-term compatibility, try a short project or exploration together first. A small, low-pressure collaboration is a great way to test the waters.

Whether you're working solo or as part of a team, the most important thing is to stay curious, keep learning, and take that first step.

You Don't Need a Supercomputer or a PhD

There are a few common misconceptions about working in deep learning for biology that are worth challenging:

You need huge budgets and compute power.

In an age of multimillion-dollar training runs for massive language models, it's easy to assume you need vast resources. But that's not always the case:

Prototype with small models

Start small to iterate quickly. You might find that lightweight models are more than enough for your goals.

Use free or affordable compute

Platforms like Google Colab and Kaggle offer free GPU access for smaller projects. For more demanding workloads, cloud providers such as Amazon Web Services (AWS), Microsoft Azure, and Google Cloud Platform (GCP) offer scalable paid instances.

Not everything is about scale

Many valuable projects focus on analyzing existing models rather than training new ones. These often require modest

compute but can yield deep insight. There's still a lot we don't understand about how deep models behave.

You need deep expertise in machine learning or biology (or both).

Another myth is that only highly trained experts can contribute meaningfully. In reality:

Better tools

Modern frameworks make it easier than ever to build and experiment with powerful models.

Open source culture

Freely available code and pretrained models let you learn from and build on existing work.

Educational resources

There's now no shortage of tutorials, videos, and walk-throughs online to help you get started.

Plenty of untapped problems

Many important biological questions remain unexplored by machine learning. You don't need a PhD or a Kaggle medal to work on them.

While the bleeding edge of research may require specialized knowledge and high-end infrastructure, there's plenty of room in this field for curiosity, creativity, and new perspectives—no supercomputer required.

TIP

As you explore this field, you'll almost certainly come across academic papers—whether you're digging into a specific method, reading related work, or looking for project ideas. Both biology and machine learning papers can feel impenetrable at first: the language is dense, the ideas are highly condensed, and there's often a lot of jargon. But remember:

- You're seeing the result of months or years of work by a team of researchers—and encountering it for the first time.
 - Reading papers is a skill, and like any skill, it improves with practice.
 - Blog posts, YouTube videos, and open source projects can also be great, more accessible ways to learn the same concepts.
-

With that background in place, let's dive into the technical foundations of this book.

Technical Introduction

We'll be using Python-based deep learning frameworks, in particular, JAX and Flax. JAX is a system for high-performance numerical computing and machine learning, and Flax is a flexible neural network library built on top of JAX. We'll motivate by explaining why we're using JAX and reviewing a few Python features that tend to come up in a lot of machine learning (ML) code. Then, we'll introduce some foundational machine learning concepts recurring throughout, with the main one being how a training loop is structured.

TIP

As mentioned in [“Prerequisites”](#), this book assumes basic knowledge of Python. If you're new to Python, check out the recommended resources listed there first.

Finally, to avoid repetition across chapters, we've created a small companion library called `d1fb` (Deep Learning for Biology), which wraps up common utilities and components. We'll reference it throughout the book.

Don't worry if some parts of this technical introduction feel unfamiliar or challenging at first. You're welcome to skim or skip ahead. Many of the concepts will become clearer when you see them in action later in the book.

Why JAX and Flax?

This book uses the JAX and Flax ecosystem. But why did we make this choice, when other options like PyTorch or Keras are more common?

First, some honesty: there is no one “best” framework. All of them can be used to build effective biological models, and many of the concepts in this book will carry over easily if you're using PyTorch or Keras.

We've chosen JAX/Flax primarily because:

Familiar NumPy API

JAX's `jax.numpy` module (commonly imported as `jnp`) offers an API that so closely mirrors standard NumPy for array manipulation and mathematical operations that NumPy `np` calls can often be directly substituted with `jnp`. This means users already proficient with NumPy can transition to JAX with a significantly reduced learning curve, leveraging their existing knowledge to quickly build and adapt code while gaining JAX's powerful transformations and accelerator support.

Functional programming encourages clarity

JAX's pure function style can reduce hidden state and make training logic more transparent. This fits well with the educational goals of the book—explicit is better than implicit.

Transformations are first-class

JAX provides powerful, composable transformations like `jit` (just-in-time compilation), `grad` (automatic differentiation), and `vmap` (vectorization) that work cleanly on Python functions. These tools simplify and unify many aspects of model training and evaluation.

JAX has gained traction in recent machine learning research, particularly for biology, physics, and large-scale models. Using it here helps you align with newer toolchains and experiment with modern practices.

Speed

JAX uses a compiler that can yield significant performance gains on specialized hardware such as GPUs (e.g., from NVIDIA or AMD) and TPUs (made by Google), making it well-suited for large-scale deep learning workloads. This compiler is based on XLA, a low-level system for optimizing numerical computations on accelerators.

That said, JAX and Flax come with trade-offs: a smaller ecosystem and APIs that evolve quickly (sometimes breaking things along the way). And while JAX can offer impressive speedups, that speed isn't exclusive to JAX/Flax. For example, [Keras](#) now supports a JAX backend, offering another option for users who prefer a higher-level API. If you're already comfortable with PyTorch, Keras, or TensorFlow, you're welcome to implement the ideas in this book using those tools—and even contribute your own version to this book's repository.

It's not necessary when you're just starting out, but over time it can be helpful to become familiar with more than one deep learning framework. Each has strengths in different ecosystems—for example, we use PyTorch in Chapter 2 to extract pretrained embeddings from a Hugging Face model, since many models on Hugging Face are primarily released and maintained in PyTorch.

WARNING

The deep learning field moves fast. While we use the `lax` API in Flax throughout this book, a newer API called `nnx` has recently emerged as the recommended way to build models. `lax` remains fully supported, but be aware that you may come across other tutorials or examples that use `nnx`, which has a slightly different syntax.

We'll introduce key JAX concepts as needed throughout the book, but we won't cover the entire library in detail. For more detailed hands-on learning, check out the official [JAX tutorials](#). And if you run into unexpected behavior, the JAX “[sharp bits](#)” [notebook](#) is an excellent reference for common gotchas and how to avoid them.

A note on performance

As this is an educational book, our focus is on clarity rather than peak performance. That means we won't cover things like precision tuning, advanced hardware strategies, or distributed training. But these things can matter a lot in real-world setups.

If you're comfortable with the basics and want to go deeper, here are a few areas worth exploring:

Numerical precision and tuning

Many machine learning operations, especially matrix multiplications (matmuls), benefit from reduced-precision formats like `bfloat16`, which can significantly improve speed and memory usage with minimal impact on model accuracy. JAX lets you control the precision used for matmuls via `jax.default_matmul_precision`, helping you take advantage of specialized hardware like Tensor Cores (on NVIDIA GPUs) or matrix units (on TPUs). Lower-precision training is widely used in large-scale setups because it enables training larger models more efficiently and cost-effectively.

Profiling tools like `jax.profiler` or TensorBoard

Profiling helps you identify where your code is spending time and memory so you can spot bottlenecks in training and optimize the most expensive operations.

Memory-efficient training techniques

Methods like gradient checkpointing (`remat` in JAX) let you trade off computation for memory, allowing you to train deeper models without running out of RAM.

Training across multiple GPUs, TPUs, or even machines allows you to scale up models and datasets that wouldn't fit on a single device.

You won't need any of these to follow this book, but they are good to be aware of and are worth exploring as you grow more comfortable with the JAX ecosystem.

Python Tips

While this book doesn't cover Python background knowledge in depth, this section highlights a few helpful Python concepts you're likely to encounter when working with machine learning code in general—and with JAX and Flax in particular.

Type annotations and docstrings

Python is a dynamically typed language, meaning you don't need to declare variable types (such as strings or integers) explicitly. Instead, types are determined at runtime, which makes the language flexible—but this flexibility can also make bugs harder to catch, especially in larger codebases. Adding *type annotations* helps mitigate this by improving readability, enabling static type checks with tools like `mypy`, and making debugging easier.

Here's a simple function that computes the mean squared error (MSE) between two NumPy arrays:

```
import numpy as np

def mean_squared_error(y_true, y_pred):
    squared_errors = (y_true - y_pred) ** 2
    return np.mean(squared_errors)
```

Here is an example of its use:

```
y_true = np.array([1.1, 0.1, 1.0])
y_pred = np.array([0.9, 0.2, 1.2])
mean_squared_error(y_true, y_pred)
```

Output:

```
np.float64(0.030000000000000002)
```

We can improve this function by adding type hints to specify that the inputs are `np.ndarray` objects and the return type is a `float`, along with a docstring to explain what the function does:

```
def mean_squared_error(y_true: np.ndarray, y_pred: np.ndarray) -> float:
    """
    Calculate the Mean Squared Error (MSE) between two NumPy arrays.

    Args:
        y_true (np.ndarray): Ground-truth values.
        y_pred (np.ndarray): Predicted values.
    """
    squared_errors = (y_true - y_pred) ** 2
    return np.mean(squared_errors)
```

These changes don't affect the function's behavior, but they offer several benefits:

Clarify input and output types

It's immediately clear that `y_true` and `y_pred` should be NumPy arrays and the return value is a `float`. Note that some machine learning code goes further by specifying the data type within the array (e.g., `arr: NDArray[np.float64]`), but we will not do this in this book.

Enhance documentation

IDEs and documentation tools can provide better inline help and autocompletion. This can really improve productivity.

Improve readability

The function is easier to understand for others (or your future self).

Enable static checking

Tools like `mypy` can catch type-related errors.

The MSE example is very simple, so adding type hints and a full docstring is arguably overkill—but the principle is important.

We won't necessarily always use typing and docstrings throughout this book due to space constraints, but they're good habits to adopt in your own projects. When we do include docstrings in the main book text, we'll usually keep them to a single line, which will save a few trees in printing.

Decorators

Decorators are functions that modify the behavior of other functions or methods. In machine learning and data science, decorators are often used to enhance performance, cache results, or log function behavior.

Just-in-time (JIT) compilation with JAX

One of the most common decorators when working in JAX is `jax.jit`, which performs JIT compilation to accelerate code execution. The first run of a JIT-compiled function is slower because it is compiled to XLA (Accelerated Linear Algebra) machine code. However, subsequent calls run significantly faster.

Suppose we have a function that takes a JAX array, raises all values to the 10th power, and sums them:

```
import jax
import jax.numpy as jnp

def compute_ten_power_sum(arr: jax.Array) -> float:
    """Raise values to the power of 10 and then sum."""
    return jnp.sum(arr**10)
```



```
arr = jnp.array([1, 2, 3, 4, 5])
compute_ten_power_sum(arr)
```

Output:

```
Array(10874275, dtype=int32)
```

We can speed up this function in one of two ways. Either directly:

```
jitted_compute_ten_power_sum = jax.jit(compute_ten_power_sum)
jitted_compute_ten_power_sum(arr)
```

Output:

```
Array(10874275, dtype=int32)
```

Or, via the Python syntactic sugar, we can apply it when defining the function with the `@jax.jit` decorator:

```
@jax.jit
def compute_ten_power_sum(arr: jax.Array) -> float:
    """Raise values to the power of 10 and then sum."""
    return jnp.sum(arr**10)

compute_ten_power_sum(arr)
```

Output:

```
Array(10874275, dtype=int32)
```

The output remains the same for all, but the jitted functions run much faster. If you are working in a Jupyter notebook, you can use `%timeit` to measure the execution time for a line of code (or `%%timeit` for entire

cells). Try timing the function with and without `@jax.jit`. On a GPU, you may see an $\sim 20\times$ speedup.

NOTE

How does `@jax.jit` work? Briefly, when you apply `@jax.jit`, JAX doesn't run your function like regular Python. Instead, it first traces the function—it runs through it once using special tracer objects (not real data) to build a computation graph. This graph is a static representation of all the numerical operations performed, with control flow unrolled and variable shapes and types fixed.

Once the graph is built, JAX compiles it using XLA (Accelerated Linear Algebra), a backend that generates highly optimized machine code. This compiled version is cached and reused whenever the function is called again with the same input shapes and types—leading to significant speedups.

JIT compilation is powerful, but it comes with trade-offs, especially for debugging. This is because:

- Python debugging tools like `print()` statements or `pdb` don't behave as expected inside jitted functions.
- Side effects (e.g., `print()`, logging, or modifying a list) are not actually executed during tracing, since JAX skips anything that doesn't affect the computation graph.
- Error messages can refer to internal JAX or XLA code instead of your original function and can be quite cryptic.

While you can temporarily disable jitting by commenting out `@jax.jit`, this becomes impractical if many functions rely on JIT. Fortunately, you can globally disable JIT by setting the environment variable `JAX_DISABLE_JIT=True`, which forces all jitted functions to run normally. This is a convenient way to debug without rewriting your code. See the [JAX debugging documentation](#) for more details.

Preconfiguring JAX jit with partial

There is a common source of confusion in machine learning code regarding usage of `partial`, especially with JAX code. The use of

`functools.partial` is to prefill (or “bind”) some arguments of a function, returning a new function with those values fixed. This is a general Python utility and is not specific to JAX or ML.

Here, we adapt the `scale` function and create a new function, `scale_by_10`:

```
from functools import partial

def scale(x, scaling_factor):
    return x * scaling_factor

# Create a new function 'scale_by_10' where 'scaling_factor' is fixed to 10.
scale_by_10 = partial(scale, scaling_factor=10)
scale_by_10(3)
```

Output:

```
30
```

Here, ``scale_by_10`` is a new function that behaves like `scale(x, 10)`.

In the context of JAX, `partial` is often used to customize a decorator before applying it, like this: `@partial(jax.jit, static_argnums=...)`. This is a way to configure the `jax.jit` decorator itself. As mentioned previously, `jax.jit` compiles your Python function for speed. However, JAX needs to know if certain arguments are static. Static arguments are typically non-JAX array types (like integers, strings, or booleans) that control the structure of the computation (e.g., in if/else statements). If a static argument changes, JAX may need to recompile the function.

Let’s say we want to compute a summary statistic over an array, choosing either the mean or the median based on a string argument `average_method`. Since this choice affects the control flow, JAX needs to know the value of `average_method` at compile time:

```

from functools import partial

import jax
import jax.numpy as jnp

@partial(jax.jit, static_argnums=(0,))
def summarize(average_method: str, x: jax.Array) -> float:
    if average_method == "mean":
        return jnp.mean(x)
    elif average_method == "median":
        return jnp.median(x)
    else:
        raise ValueError(f"Unsupported average type: {average_method}")

data_array = jnp.array([1.0, 2.0, 100.0])

# JAX compiles one version of 'summarize' for average_method="mean".
print(f"Mean: {summarize('mean', data_array)}")

# JAX compiles another version for average_method="median".
print(f"Median: {summarize('median', data_array)}")

# Calling with "mean" again uses the cached compiled version.
print(f"Mean again: {summarize('mean', data_array)}")

```

Output:

```

Mean: 34.333335876464844
Median: 2.0
Mean again: 34.333335876464844

```

If we didn't mark `average` as static with `static_argnums=(0,)`, JAX would throw an error, because it can't trace control flow that depends on strings unless it knows their value ahead of time. Marking arguments as static tells JAX to compile a separate, specialized version of the function for each unique value of that static argument it encounters.

TIP

A bit of a clarification on the meaning of “static” versus “dynamic”: JAX treats most numerical inputs (like `jax.Array`, `float`, or `int`) as dynamic, meaning they can vary between calls without requiring recompilation, as long as their shapes and types stay the same.

Other inputs, like strings, Python objects, or functions, are static: they affect control flow or can’t be traced as part of the computation graph. If you pass them into a jitted function, you must mark them as static using `static_argnums` or close over them using a closure instead (see the next section).

Closures

A closure is a function that “remembers” the environment in which it was created. This means it can access variables from its enclosing (outer) function’s scope, even after that scope has finished executing:

```
def outer_function(x):
    def inner_function(y):
        return x + y # inner_function "closes over" x.

    return inner_function

add_five = outer_function(5) # x is 5.
result = add_five(10) # y is 10.
print(f"Closure result: {result}")
```

Output:

```
Closure result: 15
```

In this example, `add_five` is a closure. It “remembers” that `x` was 5 when `outer_function` was called.

Closures are used extensively in JAX-based machine learning code. Many components—like loss functions, regularizers, and augmentation pipelines—are parameterized by configuration values. Instead of passing

these values as arguments (which might require `static_argnums` if used inside control flow), they're often closed over. A little later we will see this in action when defining the JAX training loop.

Generators

Generators are functions that allow you to iterate over data lazily, yielding one item at a time. They are especially useful for working with large datasets where loading everything into memory (RAM) at once is impractical or impossible.

Here's a simple generator function that simulates streaming batches of data:

```
from typing import Iterator

def data_generator() -> Iterator[dict]:
    """Yield data samples with features and labels."""
    for i in range(5):
        yield {"feature": i, "label": i % 2}

# Example usage.
generator = data_generator()
next(generator)
```

Output:

```
{'feature': 0, 'label': 0}
```

We will be working with TensorFlow datasets (TFDSs) in some chapters. Because JAX doesn't include a native data-loading library, it's common to see hybrid setups using TFDS. If you have your data in NumPy arrays, you can easily create a TensorFlow dataset using `tf.data.Dataset.from_tensor_slices`. This allows you to integrate NumPy data into TensorFlow pipelines for efficient training and preprocessing. It provides a clean API for batching, shuffling, and prefetching (preloading data before it's needed to increase training speed), which is helpful when getting started:

```
import tensorflow as tf

features = np.array([1, 2, 3, 4, 5])
labels = np.array([0, 1, 0, 1, 0])

# Create a TensorFlow dataset from the NumPy arrays.
dataset = tf.data.Dataset.from_tensor_slices((features, labels))

# Batch dataset with batch size of 2 and drop the final batch if incomplete.
batched_dataset = dataset.batch(2, drop_remainder=True)

# Create a dataset (ds) iterator and retrieve the first batch using next().
ds = iter(batched_dataset)
next(ds)
```

Output:

```
(<tf.Tensor: shape=(2,), dtype=int64, numpy=array([1, 2])>,
 <tf.Tensor: shape=(2,), dtype=int64, numpy=array([0, 1])>)
```

In later chapters, we will also be writing custom data pipelines that offer more control.

Anatomy of a Training Loop with JAX/Flax

While the details of machine learning projects may vary, the core structure of training a model remains fairly consistent. This core structure will serve as a foundation throughout the chapters of this book. Let's walk through the basic layout of training a model using JAX and Flax—a pattern you can build upon as you explore more complex examples.

Defining a dataset

Let's create some toy data where the target y values are a linear transformation of the x values with a bit of random noise added. We'll use the relationship $y = 2x + 1$ with Gaussian noise, as you can see in [Figure 1-1](#).

```
import jax
import jax.numpy as jnp
import matplotlib.pyplot as plt
from flax import linen as nn

# In JAX, randomness is handled explicitly by passing a random key.
# We create a key here to seed the random number generator.
rng = jax.random.PRNGKey(42)

# Generate toy data: x values uniformly sampled between 0 and 1.
rng, rng_data, rng_noise = jax.random.split(rng, 3)
x_data = jax.random.uniform(rng_data, shape=(100, 1))

# Add Gaussian noise.
noise = 0.1 * jax.random.normal(rng_noise, shape=(100, 1))

# Define target:  $y = 2x + 1 + \text{noise}$ .
y_data = 2 * x_data + 1 + noise

# Visualize the noisy linear relationship.
plt.scatter(x_data, y_data)
plt.xlabel("x")
plt.ylabel("y")
plt.title("Toy Dataset:  $y = 2x + 1 + \text{noise}$ ")
plt.show()
```

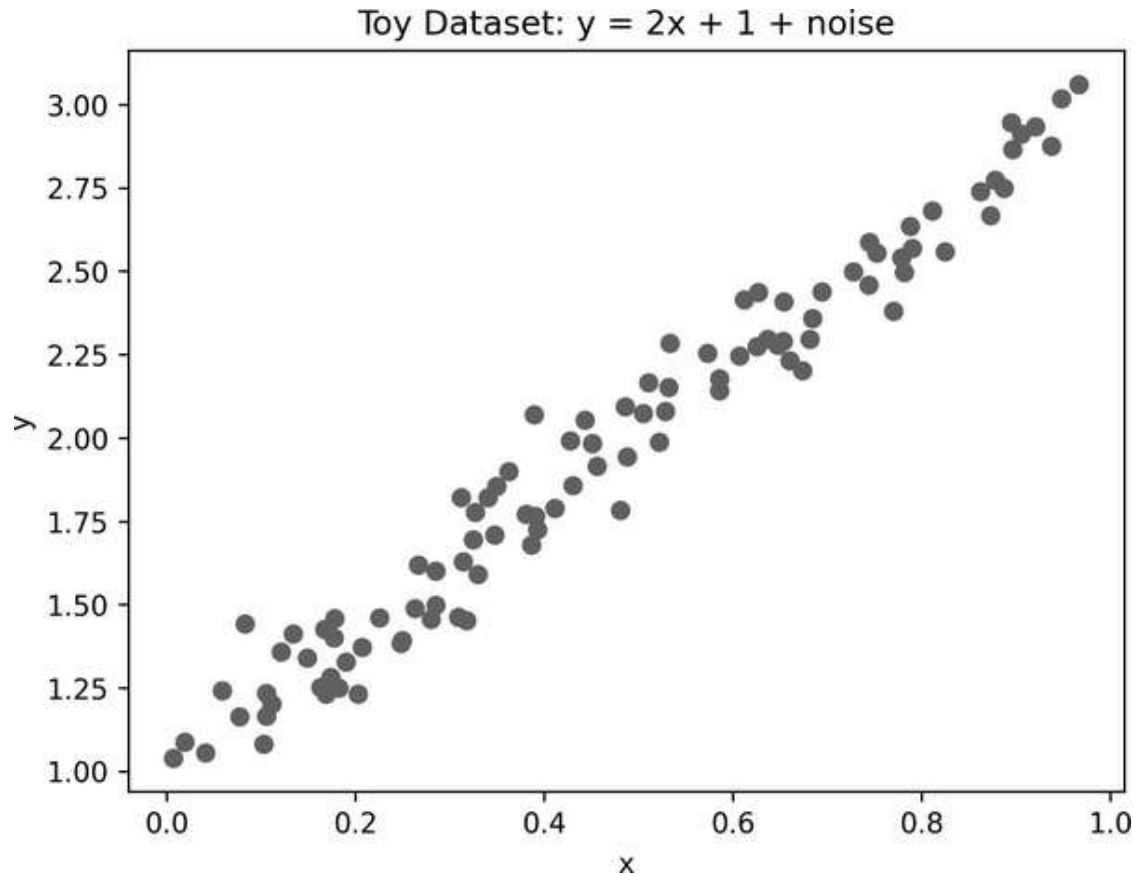



Figure 1-1. This scatterplot visualizes the underlying relationship that we want our model to learn.

Defining a model

In Flax, we define a model by inheriting from `nn.Module`. The `@nn.compact` decorator allows us to define layers directly inside the `__call__` method, rather than in the class's `setup()` method. This is especially useful for simple, sequential models.

Here's a minimal example, a single linear (dense) layer with one output unit and no activation function:

```
class LinearModel(nn.Module):
    @nn.compact
    def __call__(self, x):
        # Applies a single dense (fully connected) layer with 1 output neuron.
        # That is, it computes  $y = xW + b$ , where the output has dimension 1.
        return nn.Dense(features=1)(x)
```

We can instantiate the model like this:

```
model = LinearModel()
```

To initialize the model's parameters, use the `.init` method with a random key and a sample input. This allows Flax to infer the input and output shapes. Here, we pass in a dummy input with shape `[1, 1]`—one example (batch size of 1) with one input feature, matching the shape of our toy data:

```
rng = jax.random.PRNGKey(42)
variables = model.init(rng, jnp.ones([1, 1]))
```

This initializes the model's parameters. The result is a dictionary containing the weights and bias for the dense layer:

```
print_short_dict(variables)
```

Output:

```
{'params': {'Dense_0': {'kernel': Array([[ -0.5220277]], dtype=float32), 'bias': Array([0.], dtype=float32)}}}
```

Here:

- `kernel` is the learned weight matrix (shape `[1, 1]`, since our input and output dimensions are both 1).
- `bias` is the learned bias term added after the matrix multiplication.

Note that while the Flax API has evolved over time, the core ideas remain stable: defining model layers, initializing parameters, and inferring shapes from inputs. Moreover, even if the syntax changes, these fundamentals will carry over to nearly any deep learning framework.

Why infer shapes from input? In Flax, the shapes of the model's weight and bias parameters are actually unknown until you run data through the model. That's because Flax follows a *functional* style: layers don't store input shape information when you define them. Instead, you provide a sample input during initialization, and Flax infers the necessary parameter shapes on the fly.

Other libraries, like PyTorch or Keras, use an object-oriented style, where layers often remember input shapes internally. This can make model construction feel more automatic, but Flax's approach gives you more control and makes model behavior easier to inspect and debug, especially when working with JAX's just-in-time (JIT) compilation.

Creating a training state

A *training state* in Flax is a container that packages together everything you need for training: the model's parameters, the optimizer, and the function used to apply the model. Let's build one:

```
import optax
from flax.training import train_state

# Define an optimizer – here we use Adam with a learning rate of 1.0.
# (Note: in most real settings you'd use a smaller learning rate like 1e-3).
tx = optax.adam(1.0)

# Create the training state.
state = train_state.TrainState.create(
    apply_fn=model.apply, # The model's forward pass function.
    params=variables["params"], # The initialized model parameters.
    tx=tx, # The optimizer.
)
```

The `TrainState` object is designed to make training in Flax cleaner and more manageable. It holds everything needed for model updates:

params

The current model parameters.

`tx`

The optimizer (in this case, Adam). “Tx” is short for transformation. In Optax (JAX’s optimization library), optimizers are defined as transformations of gradients. For example, Adam transforms raw gradients using momentum and adaptive scaling.

`apply_fn`

The function that runs the model’s forward pass.

Importantly, the training state is immutable—rather than modifying it in place, each update returns a new `TrainState` with the updated parameters. This functional style is consistent with JAX’s overall design and helps keep computations pure and traceable.

TIP

Although `TrainState` is immutable and each update returns a new object, this doesn’t lead to memory issues. JAX reuses memory efficiently, especially inside `@jit`-compiled functions.

Defining a loss function

The loss function measures how close the model’s predictions are to the true targets. Here, we use MSE, which is common for regression tasks:

```
def calculate_loss(params, x, y):
    # Run a forward pass of the model to get predictions.
    predictions = model.apply({"params": params}, x)

    # Compute MSE loss.
    return jnp.mean((predictions - y) ** 2)
```

Note that `model.apply` works here because `model` was defined earlier and is available in the current scope (e.g., the same notebook or script). We don’t need to pass it as an argument, because the function is still pure—all variable model state comes from the `params` we pass in.

Here's how you would call this function with your current model and data:

```
loss = calculate_loss(variables["params"], x_data, y_data)
print(f"Loss: {loss:.4f}")
```

Output:

```
Loss: 5.2768
```

This computes the current loss by applying the model to the data using the (random) parameters stored in `state`. Since the model hasn't learned from the data yet, the loss is expected to be relatively high. As training progresses, this number should steadily decrease—which would tell us that the model is improving at its task of predicting the targets from the inputs.

Defining the training step

The training step performs a forward pass, computes the loss and gradients, and updates the model parameters. We use `jax.jit` to compile the entire step for efficiency. While JAX can run on GPUs and TPUs without it, using `jit` ensures that the code is compiled into a single optimized graph—which runs significantly faster and takes full advantage of accelerator hardware.

```
@jax.jit
def train_step(state, x, y):
    # Compute the loss and its gradients with respect to the parameters.
    loss, grads = jax.value_and_grad(compute_loss)(state.params, x, y)

    # Apply gradient updates.
    new_state = state.apply_gradients(grads=grads)

    return new_state, loss
```

Often, you don't just want the gradients (to update the parameters). You also want to log the loss explicitly (e.g., to plot it over time). But computing gradients requires computing the loss anyway. Instead of doing this twice, `jax.value_and_grad` is a convenient utility that does both at once:

- It evaluates the function you give it (in this case, `calculate_loss`) to get the loss value.
- It computes the gradients of that loss with respect to the parameters.

This avoids redundant computation.

The result of our `train_step` function is a new `TrainState` with updated parameters, along with the current loss, which we can use to monitor training progress.

Note that often you will encounter the training step defined with a closure like so:

```
@jax.jit
def train_step(state, x, y):
    def calculate_loss(params):
        # state, x and y are not part of the function signature but are accessed.
        predictions = state.apply_fn({"params": params}, x)
        return jnp.mean((predictions - y) ** 2)

    loss, grads = jax.value_and_grad(calculate_loss)(state.params)
    state = state.apply_gradients(grads=grads)
    return state, loss
```

Here, `state`, `x`, and `y` are closed over—they're not part of the `calculate_loss` function's input signature, making the code more compact and easier to read and reason about.

Handling auxiliary outputs in the loss function

As a slight aside (but one that comes up often), sometimes we want to return extra information from the loss function—for example, predictions or metrics for logging—without affecting the gradient computation. JAX makes this easy with the `has_aux=True` flag. It tells `value_and_grad` to

treat everything after the loss as “auxiliary” and to exclude it from gradient computation.

For example, let’s modify our loss function to also return predictions and accommodate this by using `has_aux=True` in `jax.value_and_grad` in our `train_step`:

```
@jax.jit
def train_step(state, x, y):
    def calculate_loss(params):
        predictions = state.apply_fn({"params": params}, x)
        loss = jnp.mean((predictions - y) ** 2)
        return loss, predictions # Return both loss and preds (aux info).

    (loss, predictions), grads = jax.value_and_grad(calculate_loss, has_aux=True)(
        state.params
    )
    state = state.apply_gradients(grads=grads)
    return state, (loss, predictions)
```

Without `has_aux=True`, JAX expects the loss function to return a single scalar. Returning anything else (like predictions) will raise an error. By setting `has_aux=True`, you’re telling JAX: “only differentiate with respect to the loss; ignore any extra outputs, like predictions.”

Defining the training loop

Now that all the components are in place, we can define the training loop that actually updates the model parameters based on the data.

In most machine learning workflows, training happens over either *steps* or *epochs*:

- A *step* refers to one update of the model using a single batch of data.
- An *epoch* is a full pass through the entire training dataset—typically consisting of many steps. In our toy example, we feed the entire dataset (100 input–output pairs) into the model at once, without batching. This means each step is equivalent to a full epoch. In more realis-

tic settings, you'd typically break the data into batches, resulting in many steps per epoch.

Let's now train the model:

```
num_epochs = 150 # Number of full passes through the training data.

for epoch in range(num_epochs):
    state, (loss, _) = train_step(state, x_data, y_data)
    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss:.4f}")
```

Output:

```
Epoch 0, Loss: 5.2768
Epoch 10, Loss: 0.9498
Epoch 20, Loss: 0.1091
Epoch 30, Loss: 0.0845
Epoch 40, Loss: 0.0283
Epoch 50, Loss: 0.0258
Epoch 60, Loss: 0.0106
Epoch 70, Loss: 0.0105
Epoch 80, Loss: 0.0106
Epoch 90, Loss: 0.0102
Epoch 100, Loss: 0.0101
Epoch 110, Loss: 0.0100
Epoch 120, Loss: 0.0100
Epoch 130, Loss: 0.0100
Epoch 140, Loss: 0.0100
```

The model converges quickly—its loss decreases rapidly to a stable, low value. After training, we can test how well the model has learned the underlying pattern by comparing its predictions to the true target values, as shown in [Figure 1-2](#):

```
# Generate test data (x values between 0 and 1).
x_test = jnp.linspace(0, 1, 10).reshape(-1, 1)
y_test = 2 * x_test + 1 # Ground truth: linear function without noise.

# Get model predictions.
```



```

y_pred = state.apply_fn({"params": state.params}, x_test)

plt.scatter(x_test, y_test, label="True values")
plt.plot(x_test, y_pred, color="red", label="Model predictions")
plt.xlabel("x")
plt.ylabel("y")
plt.legend()
plt.title("Linear Model Predictions vs. True Relationship")
plt.show()

```

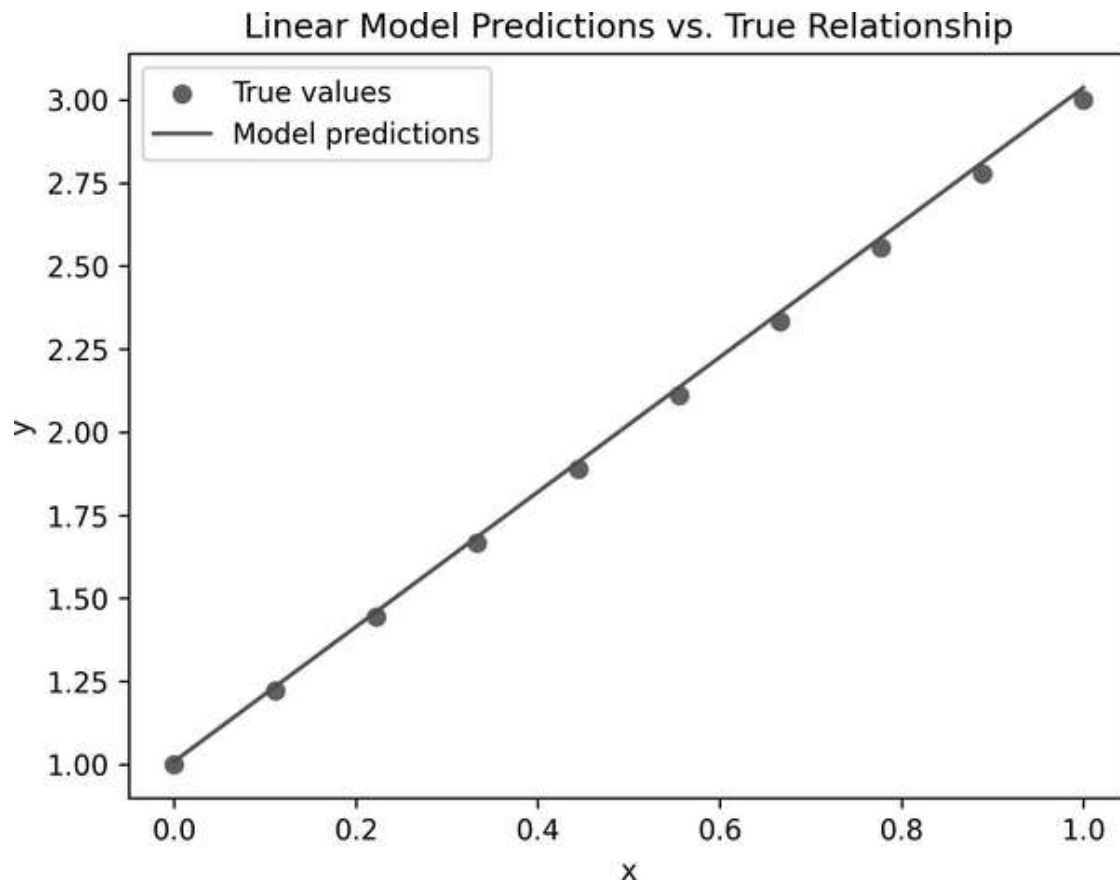


Figure 1-2. Scatterplot comparing the linear model predictions with the true values.

We can see that after training, the model has learned to approximate the true function $y=2x+1$ very closely. The predicted outputs are nearly identical to the expected values, which is exactly what we want in this toy regression task.

This may be a simple example, but it captures the core structure of almost every deep learning workflow: define a model, compute a loss, update parameters, and repeat. While real projects involve more complexity—batching, data pipelines, regularization, metrics, logging, and so on—the basic loop you’ve built here is the foundation of it all. You now have a

solid mental model for how training works in JAX and Flax—and the foundation to build much more powerful systems.

TIP

Where to go from here? Once you’ve built a working training loop (an important milestone), it’s common to expand in several directions: add metrics that capture key aspects of model performance, split off a validation set to check generalization, and track progress over training time. These are all building blocks of real-world deep learning workflows, and we’ll walk through them in upcoming chapters.

Machine Learning Tips

Here’s a brief reminder of some machine learning concepts, focusing on what we use throughout the book. We explain these ideas in more detail as necessary when they come up.

Types of Tasks

In *classification*, the model predicts a label or a probability distribution over labels. There are three types of classification:

Binary classification

Choose between two options; for example, whether a cell is healthy or not.

Multiclass classification

Choose one label out of several; for example, based on patterns in a biological sample, a model might predict which part of the body it came from—like the brain, liver, or skin. Each sample belongs to exactly one class.

Multilabel classification

Choose all labels that apply, not just one. For example, when analyzing a cell image, the model might predict which structures are

visible—such as the nucleus, membrane, and mitochondria. In this case, multiple labels can be true for the same image.

Regression involves predicting a continuous value, such as the binding strength between two molecules.

We also use *representation learning*—learning useful embeddings or feature representations without direct supervision. These embeddings capture patterns or structure in the data, which can then be used for tasks like clustering, visualization, or as inputs to downstream models.

Types of Architectures

In this book, we primarily use the following model architectures:

Linear models and multilayer perceptrons (MLPs)

These are fully connected networks that transform input vectors into output vectors through stacked dense layers. They are simple and widely used.

Convolutional neural networks (CNNs)

CNNs apply spatial filters to model local structure, typically mapping images to images or feature maps. They are especially effective for image and sequence data.

Transformers

Model long-range dependencies in sequences using attention. They operate on sets or sequences and are now state of the art in many areas of biology and language modeling.

Graph neural networks (GNNs)

Operate on graph-structured data, passing messages between nodes and their neighbors. They are useful when data has relational or interaction-based structure.

Autoencoders

Learn compact representations by encoding inputs into a latent space and reconstructing them. They are common in unsupervised learning and denoising tasks..

We'll explain each of these as they appear throughout the book.

INDUCTIVE BIASES

Modern deep learning architectures are increasingly mature. Rather than trying to invent the next transformer, you'll often get further by composing existing building blocks in thoughtful ways, guided by domain knowledge.

Think about what *inductive biases* (assumptions about the structure of the data) make sense for your problem: Are nearby data points related (like the pixels in images)? Is there a sequential structure (like in DNA or text)? Do entities interact in a graph-like way? These structural assumptions help your model learn more efficiently by narrowing the search space of possible functions.

A common type of inductive bias is an *invariance*—where the model assumes that certain transformations shouldn't affect the output. For example:

- CNNs assume *translation invariance*: shifting an image slightly shouldn't change the prediction.
- GNNs assume *permutation invariance*: the order of nodes in a graph shouldn't affect the output.
- Transformers are *permutation invariant* by default: they ignore input order unless positional encodings are added. This gives them flexibility, but it also means they rely on you to encode any sequential structure.

Dataset Splits

Machine learning models are typically trained using three dataset partitions:

Training set

Used to fit the model's parameters (its weights) by minimizing a loss function

Validation set

Used to tune *hyperparameters* (discussed next) and evaluate performance during development

Test set

Held out until the end to assess the model's final performance on truly unseen data, providing an estimate of how well it generalizes

This split structure is good practice because it helps ensure your model generalizes well to new data and gives you a reliable estimate of its real-world performance.

Hyperparameters

Hyperparameters control how a model is trained. They're set before training begins and are not updated by the optimizer. Common examples include:

Learning rate

How quickly the model updates its weights

Batch size

How many examples are processed together during one update

Model size

The number of layers, hidden units, or attention heads

Regularization

The dropout rate or weight decay, to prevent overfitting

We evaluate different hyperparameter settings using performance on the validation set. If we tune hyperparameters based on training perfor-

mance alone, we risk *overfitting*—the model may simply memorize the training data, including noise or outliers.

Overfitting leads to poor performance on new data. In contrast, *generalization* means the model has learned patterns that apply beyond the training examples. This is a key requirement in biological applications, where test data can often come from different experiments or conditions.

Activation Functions

Activation functions introduce nonlinearity into neural networks, enabling them to model complex relationships between inputs and outputs.

As data flows through a model, it's transformed into intermediate tensors known as *activations*. These activations pass through a series of linear projections and nonlinear activation functions at each layer. The output of the final hidden layer is then often passed through a final activation function, which produces the model's ultimate prediction.

Choosing the right final activation is important—it should reflect the kind of data you're trying to predict. For example:

- Use `sigmoid` for binary classification, where outputs should lie between 0 and 1.
- Use `softmax` for multiclass classification, where outputs represent probabilities across categories.
- Use *no* final activation if your loss function expects raw logits (e.g., `sigmoid_cross_entropy` or `softmax_cross_entropy`). These loss functions apply the activation internally, so applying it yourself would be redundant or even harmful.
- Avoid ReLU or GELU as final activations when predicting real-valued outputs that can be negative—they will clip or distort negative values.
- For regression tasks, no activation is often best—or use `tanh` or `sigmoid` only if your target values are known to be bounded (e.g., between -1 and 1 or 0 and 1).

When in doubt, check the range of your target values, and choose an activation (or none) that allows the model to produce outputs in that range.

The term *logits* is often used to describe the raw, unnormalized outputs of the final layer, especially before applying softmax or sigmoid. But it's a somewhat overloaded term that is sometimes used more loosely—for example, to describe intermediate values passed into a softmax, such as attention scores in a transformer.

The following are the most commonly used activation functions, with their shapes shown in [Figure 1-3](#):

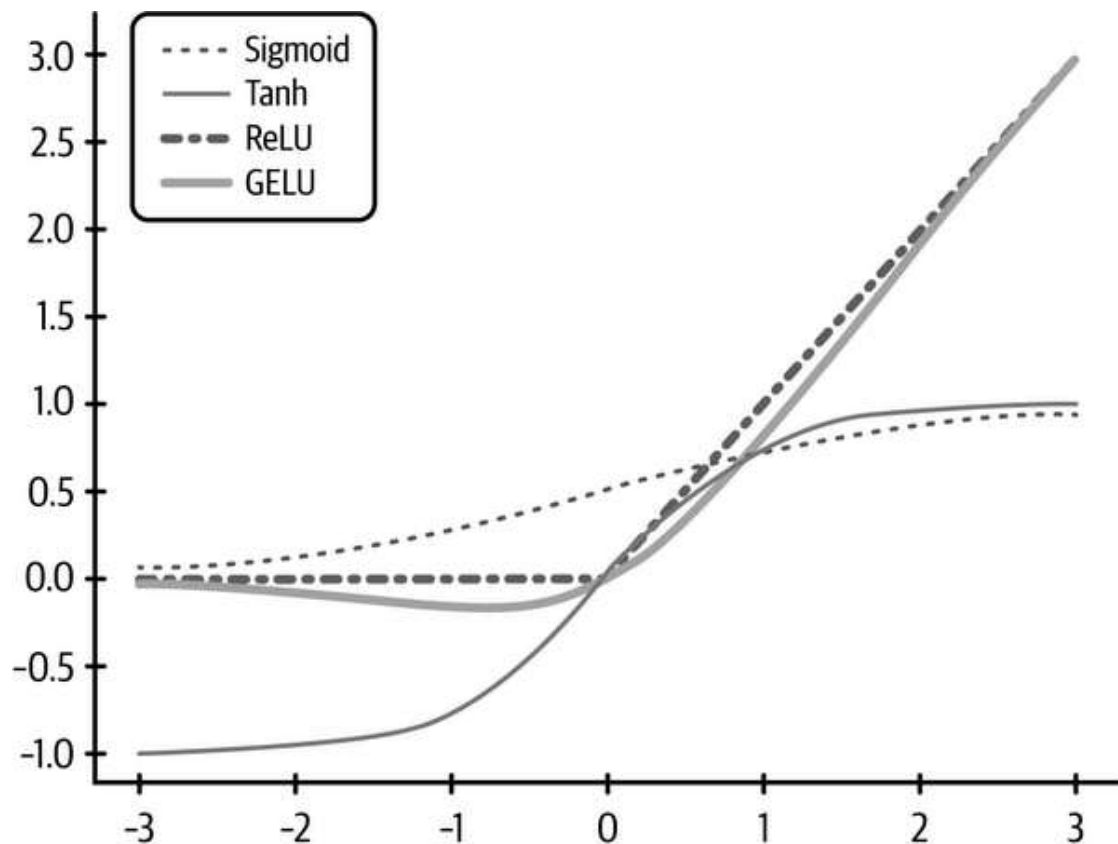


Figure 1-3. Common activation functions used in deep learning.

A bit more detail on these common activation functions:

ReLU (rectified linear unit)

Outputs zero for negative inputs and the input itself for positive inputs. It is simple and effective, especially in deep networks.

GELU (Gaussian error linear unit)

A smoother alternative to ReLU, often used in transformer models. It can yield slightly better results.

Sigmoid and tanh

Older activation functions that squash values into fixed ranges.

Sigmoid maps inputs to a range between 0 and 1, while tanh maps to a range between -1 and 1. They're useful in certain settings, such as output layers for binary classification, but can suffer from gradient issues in deeper models.

Softmax

Converts a vector of values into a probability distribution that sums to 1. Used in the final layer of multiclass classification models. Not applied element-wise; it operates across the whole vector.

In this book, we'll usually stick to ReLU or GELU for hidden layers, and choose the final activation based on the prediction task.

Optimizers

Optimizers are algorithms that adjust a model's parameters (its weights and biases) to reduce the error (loss) during training. They do this using *gradient descent*, which computes how each parameter affects the loss and updates it to reduce the error.

We mostly use *Adam*, a widely used optimizer that adapts the learning rate for each parameter and combines ideas from momentum and RMSProp. It usually trains faster and more reliably than plain gradient descent, especially in noisy or sparse settings.

Initialization Strategy

Before training starts, we need to set initial values for model parameters. This step is more important than it sounds. Poor initialization can cause gradients to vanish or explode, making training unstable.

We typically use Xavier (Glorot) initialization, which is designed to keep the scale of activations and gradients roughly stable across layers. This helps training proceed smoothly. By default, Flax uses Xavier initialization for most layers like `Dense` and `Conv`, so you don't usually need to specify it manually.

Model Checkpointing

During training, it's often useful to periodically save your model parameters so that you can resume later or avoid losing progress. While production training pipelines often use robust checkpointing strategies (e.g., saving best-performing checkpoints, versioned histories, etc.), that level of complexity isn't necessary for the teaching examples in this book.

Instead, we provide a lightweight utility that saves and restores just the most recent checkpoint. This is enough to pause and resume training or to store the final model output for later use. You'll see it used in several chapters to simplify experimentation and reduce boilerplate.

If you're building your own training loop for production or research, consider upgrading to a more complete checkpointing solution—for example, using [Orbax](#), Flax's newer checkpointing system.

Early Stopping

In real-world training, it's often a good idea to stop training when your validation performance stops improving. This is known as *early stopping*, which helps prevent overfitting and saves compute.

In this book, we often show longer training runs to visualize how the loss evolves over time. But in practice, you'll likely want to use early stopping when training your own models.

Flax includes a simple utility for this:

```
from flax.training.early_stopping import EarlyStopping
```

You can track validation metrics and stop training when they stop improving for a certain number of steps (the `patience` parameter).

Selecting a Working Environment

Training large neural networks often benefits from *GPU acceleration*. There are a few ways to access a GPU, depending on your budget, goals, and technical comfort level.

One option is to use a local machine with a GPU, such as a gaming desktop or a workstation you've set up yourself. This gives you full control and avoids ongoing cloud costs, but it requires up-front hardware investment and setup. Another option is to rent GPU access from cloud providers like AWS, GCP, or Azure. These offer flexibility and scalability but can get expensive over time, especially if you're training large models.

For many beginners and small projects, Google Colab is a great place to start. It provides free, cloud-based Jupyter notebooks with GPU or TPU support and minimal setup.

In the following sections, we'll briefly walk through your options, from interactive notebooks to fully customized GPU development environments.

Selecting an Interactive Notebook

Jupyter notebooks are a popular and powerful tool for interactive coding, making them ideal for running the code examples in this book. They let you write code, execute it in cells, visualize results, and document your work, all in one place. This interactivity makes it easy to experiment, debug, and iterate quickly. Common notebook environments include JupyterLab, VSCode with Jupyter extensions, and Google Colab.

Google Colab in particular provides cloud-based notebooks with free access to GPUs and TPUs, making it a great option if you don't have a powerful local machine. It runs entirely in your browser and requires no setup beyond a Google account. You can install libraries with commands like `!pip install jax flax optax` and save notebooks directly to Google Drive. To enable hardware acceleration, navigate to the Runtime menu.

From the drop-down menu, select “Change runtime type” and then select a GPU or TPU.

We provide all code in this book as Google Colab notebooks that you can open, run, and modify interactively.

WARNING

Colab sessions can time out after periods of inactivity, so remember to save your work frequently.

While notebooks are excellent for exploration and prototyping, they can make version control, debugging, and maintaining complex projects more difficult. For longer-term work, setting up a dedicated development environment with GPU support offers more control, scalability, and reproducibility.

Structuring Your Code for Reuse and Debugging

Notebooks are great for interactive use, but as you begin building your own projects, it’s worth thinking about how you will structure it early. Organize your code into clear, reusable Python modules: one for datasets, one for models, one for metrics and evaluation. Keeping these pieces separate makes your code easier to debug, scale, and reuse later.

This modular approach mirrors how this book is structured. Each subsection of each chapter generally focuses on a specific building block: dataset, model, training loop, and so on. As you follow along, you might try using this structure in your own code. Build clean components you can plug into future projects, and you’ll move faster each time.

A good dataset class in particular will help you catch issues early. There are plenty of reasons you might get confused during a project. If your dataset and metrics are cleanly separated and easy to inspect, you can at least rule them out as the source of the problem.

TIP

Add frequent sanity checks. Can your model overfit a tiny dataset (like 10 examples)? Do loss and accuracy behave as expected when you shuffle labels?

Plot everything. Visualizing predictions, losses, or inputs will often reveal issues you wouldn't notice in the final numbers alone—like shifted targets, mislabeled data, empty tensors, or models that haven't learned anything at all.

Setting Up a GPU Development Environment

As your models grow more complex, you may want to move beyond notebooks into a more powerful GPU-enabled setup, either locally or in the cloud. A robust development environment can speed up experimentation, simplify debugging, and make your workflow more reproducible.

For local development, we recommend the following:

Use Docker with NVIDIA Docker to create containerized environments that seamlessly access your GPU.

Pair it with an editor like VSCode, which supports Docker and remote development for a smooth workflow.

Use Git for version control to track changes, collaborate, and back up your work.

Mac users with Apple Silicon can try `jax-metal` to enable GPU acceleration via Apple's Metal backend. While it's improving rapidly, not all features are fully supported yet, so expect occasional compatibility issues.

TIP

Paid cloud services like AWS, GCP, or Azure offer on-demand GPU instances if you don't have a local GPU. Alternatives like Paperspace, Lambda Labs, or RunPod can offer simpler setup and better value for smaller-scale projects.

Version Conflicts

Machine learning and scientific computing libraries evolve quickly—and not always in sync. You’ll inevitably hit version conflicts between tools like NumPy, JAX, Flax, PyTorch, or Hugging Face Transformers. Strive for a balance: use versions recent enough to benefit from improvements, but not so new that they break compatibility with everything else.

Tools like `uv`—a faster, more flexible alternative to `pip`—can help override incompatibilities and install packages even when their metadata says they’re not compatible. It’s not a permanent solution, but it can get you unstuck.

If you’re working outside a notebook, always use a virtual environment (`venv`, `conda`, or `uv venv`) to keep dependencies isolated. Another option is to define your environment in a Docker container, which guarantees full reproducibility across machines. This setup is especially useful when working on remote GPU instances or in the cloud—something we’ll explain later in the book.

When in doubt, check GitHub issues or forums—version mismatches are a common pain point for everyone, and at the very least, you can commiserate with others.

If you’re working in a notebook, version issues can be especially tricky. Google Colab, for example, comes with many preinstalled packages—but these can be outdated or incompatible with the latest JAX/Flax stack. You can install or override versions directly in a cell using `!pip install`, but you may need to restart the runtime afterward for changes to take effect (Runtime > Restart runtime).

A Living Document

While this book endeavors to capture the state of the field at the time of writing, deep learning and biology remain fast-moving frontiers. A printed book is a static artifact; in contrast, frameworks evolve, APIs break, and new ideas emerge constantly.

NOTE

We've done our best to future-proof examples and flag potential breaking changes (like Flax's transition from `linen` to `nnx`). Still, some discrepancies may arise over time. If you spot anything out of date—or have corrections, improvements, suggestions, or extensions—please let us know. The online repository can evolve even if the book is in a fixed state.

We also encourage you to look beyond these pages. Resources like [D2L.ai](https://d2l.ai), fast.ai, the JAX ecosystem, and preprints on bioRxiv and arXiv are excellent places to go deeper.

Most of all, experiment, build, break things, train bad models, and learn. The best way to grow in deep learning for biology is to get your hands dirty.

Table of contents collapsed